

Step 3 — Docker + Rate Limiting — Hints

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 3 depth levels: **Basic**, **Intermediate**, **Advanced**.

- [Hint 1 — The Dockerfile, and where secrets actually go](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The Dockerfile, and where secrets actually go

Basic Version

A `Dockerfile` describes how to build a self-contained image of your app: start from a Python base image, copy your code in, install your dependencies, and say what command starts the server.

`docker build` builds it; `docker run` starts a container from it.

The one rule that matters most here: your `.env` file (with `OPENAI_API_KEY` in it) never gets copied into the image. Secrets go in at `docker run` time, as environment variables, not baked into the image itself.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

A minimal Dockerfile: `FROM python:3.11-slim`, `WORKDIR /app`, `COPY requirements.txt .` then `RUN pip install -r requirements.txt` (in that order, *before* copying the rest of your code — Docker caches each layer, so code changes don't force a full dependency reinstall on every build), `COPY . .`, then `CMD ["uvicorn", "api.main:app", "--host", "0.0.0.0", "--port", "8000"]`.

`.dockerignore` is `.gitignore`'s Docker equivalent — list `.env`, `.venv`, `__pycache__`, `*.db`, `.git` in it, so none of that ever gets copied into the image in the first place, even by accident from a stray `COPY . . .`.

Pass secrets at run time: `docker run --env-file .env -p 8000:8000 contentforge-api` (or `-e OPENAI_API_KEY=...` for one variable at a time). `docker history <image>` shows every layer's build command — run it after building, and confirm no layer shows your actual key value.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

The basic in-memory rate limiter almost everyone writes first — "count requests per client in the last N seconds, reject over the limit" — has a real, well-known flaw if implemented as a naive **fixed window**: reset a counter every 60 seconds, and a client can send its full limit right *before* the window resets, then its full limit again right *after* — twice the intended rate, concentrated in the few seconds spanning the boundary. That's not a hypothetical edge case; it's the first thing anyone actually trying to abuse a fixed-window limiter will find.

A **sliding window** (or a token bucket) fixes this by tracking *when* each request happened, not just *how many* happened since the last reset — reject a new request if there are already `limit` requests recorded within the last `window_seconds`, regardless of where the current moment sits relative to any fixed boundary. It's a small amount of extra bookkeeping (a list or deque of timestamps per client, instead of one integer) for a real correctness improvement.

The other real piece: what does a rejected request actually get back? A bare `429` with no explanation isn't as useful to a well-behaved client as one that also says *when to try again*.

The extra pieces:

- Track a `deque[float]` of recent request timestamps per client key (IP address is fine for this project), not a single counter that resets on a timer.
- On each request: drop timestamps older than `window_seconds` from the front of the deque, then check `len(deque) >= limit` — reject if so, otherwise append the current timestamp and allow it.
- Return a `Retry-After` header on the `429` response, computed from the oldest timestamp still in the window — tells a well-behaved client exactly how long to wait, instead of it guessing.

Sketch the sliding-window check yourself before checking Hint 2.

Difference between Basic, Intermediate, and Advanced: Basic and Intermediate both get a Dockerfile working and add *some* rate limit, generally a fixed-window counter. Advanced replaces the counter with a real sliding window, closing the boundary-burst gap a fixed window allows, and makes the rejection itself more useful to whoever's calling the API.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

Dockerfile:

```
start from a python base image
copy requirements.txt, install dependencies
copy the rest of the app code
set the start command to run uvicorn
```

.dockerignore:

```
.env, .venv, __pycache__, *.db, .git
```

rate limiting (in routes.py):

```
keep a dict: client -> count of requests seen recently
on each request: increase the count
if count is over the limit: reject with 429
reset the counts every so often (e.g. every 60 seconds)
```

test:

```
docker build the image, docker run it with --env-file .env
docker history <image>, confirm no secret value appears in any layer
run it with a required env var missing, confirm it fails to start clearly
hit /run-task quickly several times in a row, confirm a 429 eventually
```

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Dockerfile:

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
CMD ["uvicorn", "api.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

.dockerignore:

```
.env
.venv
__pycache__/
*.db
.git
```

api/rate_limit.py:

```
_request_counts: dict[str, int] = {} # reset on a timer

def check_rate_limit(client_id: str, limit: int = 10) -> None:
    _request_counts[client_id] = _request_counts.get(client_id, 0) + 1
    if _request_counts[client_id] > limit:
        raise HTTPException(status_code=429, detail="Too many requests")
```

routes.py:

```
@router.post("/run-task")
def run_task(request: TaskRequest, http_request: Request):
    check_rate_limit(http_request.client.host)
    ...
```

test:

```
docker build -t contentforge-api .
docker run --env-file .env -p 8000:8000 contentforge-api
docker history contentforge-api # confirm no secret value in any layer
docker run -p 8000:8000 contentforge-api # no --env-file: confirm a clear s
for i in 1..15: curl -X POST localhost:8000/run-task ... # confirm a 429 ki
```

Write it yourself, then compare against the [Solution](#).

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Here's almost the sliding-window limiter — fill in the missing cleanup line yourself:

```
import time
from collections import defaultdict, deque
from fastapi import HTTPException

WINDOW_SECONDS = 60
MAX_REQUESTS_PER_WINDOW = 10
_request_times: dict[str, deque[float]] = defaultdict(deque)

def check_rate_limit(client_id: str) -> None:
    now = time.monotonic()
    timestamps = _request_times[client_id]

    # your turn: remove timestamps older than WINDOW_SECONDS from the
    # left side of the deque before checking its length
    ...

    if len(timestamps) >= MAX_REQUESTS_PER_WINDOW:
        retry_after = int(WINDOW_SECONDS - (now - timestamps[0]))
        raise HTTPException(
            status_code=429,
            detail="Too many requests",
            headers={"Retry-After": str(max(retry_after, 1))},
        )

    timestamps.append(now)
```

Fill in the cleanup step, then compare all 3 of your finished versions against the [Solution](#).

Difference between Basic, Intermediate, and Advanced: same Dockerfile and rate-limiting goal at 3 completeness levels — Basic and Intermediate use a counter that resets on a timer, which allows a burst right at the reset boundary. Advanced tracks actual timestamps in a rolling window, closing that gap, and tells a rejected client exactly how long to wait via `Retry-After` instead of just a bare `429`.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)